

Lab Practice 28 May 2026

Case Study 1 - Bank Token Management System

Problem Scenario

A bank uses a token system to serve customers in the order they arrive. Each customer receives a token number and waits until their turn comes. The bank needs a system that supports adding a new customer, serving the next customer, checking the next customer to be served, and displaying all waiting customers.

Task:

Implement a Queue for the bank token system using:

1. Array-based queue
2. Linked-list-based queue

Your program should support the following operations:

- enqueue() - add a new customer token
- dequeue() - serve the next customer
- peek() - display the next customer without removing
- isFull() - check whether the circular queue is full
- isEmpty() - check whether the queue is empty
- display() - show all waiting customers

Requirements:

- In the array implementation, handle the condition when the queue is full.
- In the linked-list implementation, dynamically create nodes for new customers.
- Clearly show the difference between array-based and linked-list-based queue implementations.

Case Study 2 - Printer Job Scheduling System

A computer lab has one printer shared by many students. Print jobs arrive continuously and are processed one by one. Since the printer buffer has limited space, the system uses a Circular Queue to reuse empty spaces after jobs are printed.

Each print job has a job ID and the number of pages.

Task:

Implement a Circular Queue for the printer job scheduling system using:

1. Array-based circular queue
2. Linked-list-based circular queue

Your program should support the following operations:

- enqueue() - add a new print job
- dequeue() - process and remove the next print job
- peek() - show the next print job
- isFull() - check whether the circular queue is full
- isEmpty() - check whether the circular queue is empty
- display() - show all pending print jobs

Requirements:

- In the array implementation, use circular movement with:
 $\text{rear} = (\text{rear} + 1) \% \text{size}$
 $\text{front} = (\text{front} + 1) \% \text{size}$
- In the linked-list implementation, make the last node point back to the first node.
- Explain how a circular queue avoids wasted space compared to a simple array queue.